Phase	Theme	Code name	Code definition	Participants falling in this code	Number of participants	Example Quote
Initial setup	Discovery and guidance	External guidance required	The participant had to seek information outside Git/GitHub's immediate workflow to discover or complete commit signing.	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22	22	P9 — “The hardest part is discovering how to sign a commit in the first place. The process isn't self-contained within Git; you need to go out of your way to find, install, and configure external tools like GnuPG before you can even begin.”
Initial setup	Information sources	GitHub official documentation consulted	The participant consulted GitHub's official documentation while setting up or troubleshooting Git commit signing.	P3, P4, P5, P7, P8, P10, P11, P12, P13, P14, P15, P16, P17, P18, P19, P20	16	P14 — “However, the only information source that I used to fully complete this project was GitHub’s official tutorial on commit signatures.”
Initial setup	Information sources	Third-party blog or tutorial consulted	The participant consulted a non-official blog or written tutorial for guidance on Git commit signing.	P1, P2, P6, P18, P20	5	P1 — “This took me to a blog post by freecodecamp that explained what it is and how to set it up for yourself. I followed the steps on the guide and it worked seamlessly.”
Initial setup	Information sources	Stack Overflow or community Q&A consulted	The participant consulted Stack Overflow or another community question-and-answer source for troubleshooting or guidance.	P7, P8, P18	3	P7 — “However, I was generally on my own for digitally signing the commit. This was the most difficult part for me, since I had little guidance until I googled how to sign commits, which lead me to Stack Overflow and eventually to GitHub.”
Initial setup	Information sources	GitLab official documentation consulted	The participant consulted GitLab's official documentation for Git or commit-signing guidance.	P4, P5, P19	3	P4 — “I used GitHub’s documentation for creating an ssh key and GitLab’s documentation for setting up signed commits using ssh keys. Both were very straightforward to use and I had no issues with them.”
Initial setup	Information sources	Git-SCM documentation consulted	The participant consulted documentation from the official Git-SCM website.	P9, P12	2	P9 — “This article was very helpful and walked me through all the necessary steps from key generation to verification.”
Initial setup	Information sources	YouTube tutorial consulted	The participant used a YouTube video tutorial for setup or troubleshooting guidance.	P3, P6	2	P6 — “For tools I used, I followed the VS code Youtube Tutorial for signing commits. This tutorial video was awesome, as it was short but contained all the information I needed.”
Initial setup	Information sources	AI tool consulted	The participant used ChatGPT, an AI-generated search overview, or another AI tool for setup or troubleshooting guidance.	P2, P17	2	P2 — “I used ChatGPT to help me implement the signature in Git, as I needed guidance on which Git commands to use.”
Initial setup	Discovery and guidance	Guidance enabled smooth setup	A guide or tutorial made the setup proceed smoothly once located.	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22	22	P1 — “I immediately googled what digital key signing is on GitHub and how it works. This took me to a blog post by freecodecamp that explained what it is and how to set it up for yourself. I followed the steps on the guide and it worked seamlessly. I installed GPG to generate my keys. I generated them by doing ‘gpg –ful-gen-key’. I followed the prompting and generated an “RSA and RSA” key, selected the size as 4096 bits, and set it to never expire. I input my name and secure password to verify the key. I then exported my public and private key to a file (1 file for each key). Once I had the public key file, I turned on commit signing in my git config and then I went to my GitHub settings and added a new GPG key with the contents of the public key file.”
Initial setup	Overall setup friction	At least one initial-setup pain point	The participant reported at least one technical, configuration, environment, or tooling problem while initially setting up or verifying commit signing.	P2, P4, P5, P6, P7, P8, P10, P11, P13, P14, P16, P17, P18, P19, P22	15	P19 — “Unfortunately this did take a bit of trial and error, so ignore the commits between the initial and final ones.”
Initial setup	Configuration barriers	allowedSignersFile schema mismatch	Git's trusted-signer file required a field order or structure that differed from the standard SSH public-key file.	P2, P4, P5, P7	4	P2 — “Another verification of the signature with git log –show-signature revealed a new error: the message was signed, but there was an issue with the structure of the public key. After some troubleshooting, I realized that allowedSignersFile should have the format <email> <key_type> <key>, not the <key_type> <key> <email> format in the id_rsa.pub file. After changing the file, a final check verified that the signature was correct.”
Initial setup	Configuration barriers	Signing identity email mismatch	The email used for key generation did not match the Git/GitHub account identity required for successful verification.	P11, P16	2	P11 — “I also had an issue with the github account not having the same email as the one I used for gpg, so I had to repeat the process but this time with the github email.”
Initial setup	Configuration barriers	macOS GPG pinentry failure	GPG could not display or invoke the passphrase prompt on macOS, requiring a pinentry-specific workaround.	P6, P8, P17, P18	4	P6 — “For tools I used, I followed the VS code Youtube Tutorial for signing commits. This tutorial video was awesome, as it was short but contained all the information I needed. I also had to find a forum for gpg, as there is a known bug with macOS GPG GUI. This was a little challenging to read and ultimately find the solution I needed, but it worked eventually.”
Initial setup	Configuration barriers	Git signing key configuration omitted	Git did not use the generated key until user.signingkey, gpg.format, or an equivalent signing configuration was set.	P5, P7, P8, P10, P13, P18	6	P10 — “First I installed gnupg (I thought the package would be called gpg but apparently not) then I generated a GPG with “gpg --full-generate-key” . At this point I tried committing which of course didn’t work because Github had no public key to check the signature with so I used “gpg --list-secret-keys --keyid-format=long” to get the key ID. Using the id I got the public key which I copied to Github. Again I tried to commit because I thought since I used the same name and email as my git config I wouldn’t need to specify the key to git but that didn’t work so I used “git config --global user.signingkey” to add the ID to my git config. Finally I committed and pushed and saw that the commit was listed as signed.”
Initial setup	Tool integration	Client or platform incompatibility	A selected client or platform could not perform signed commits and forced a workflow or tool change.	P14, P18, P22	3	P14 — “I first created the repository using the GitHub desktop application, as I always do when working with GitHub, and created the text file with Windows Notepad. However, the GitHub desktop app does not support working with keys and signed commits, so I was forced to switch to using the terminal and Windows Powershell for the remainder of the process.”
Initial setup	Perceived usability	Setup straightforward after instruction	The participant characterized the setup as straightforward or easy after obtaining usable instructions.	P1, P2, P3, P4, P6, P7, P10, P12, P13, P14, P15, P16, P17, P19, P20, P22	16	P6 — “None, it was pretty straight forward after watching the tutorial.”
Initial setup	Learning outcome	Understanding of signing increased	The setup activity improved understanding of the signing workflow, cryptography, or security properties.	P21, P22	2	P21 — “I’m very familiar with Git and GitHub (my account is over seven years old), but I had never used GPG before, so this was a new experience. In the past, I relied on GitHub Desktop without understanding the underlying cryptography. This project helped bridge that gap by making the signing workflow and its security properties much clearer.”
Cross-phase	Improvement requests	GUI or setup wizard requested	The participant requested a graphical interface, setup wizard, or guided integration for signing configuration.	P2, P4, P13, P14, P16, P19	6	P4 — “The main thing I would change is to make the entire setup process a part of the git CLI, like a setup wizard of sorts, rather than using multiple tools.”
Cross-phase	Improvement requests	Consolidated documentation requested	The participant requested one complete resource that covers the full process and likely errors.	P3, P5, P7, P11, P13, P20, P21	7	P20 — “However, if there were not online blogs and resources guiding me through the process, I would prefer there to be a singular resource on GitHub that completely guides you through the process and any errors that may be encountered.”

Cross-phase	Improvement requests	Actionable diagnostics requested	The participant requested clearer error messages, prerequisite checks, or concrete remediation guidance.	P8, P22	2	P8 — “I would’ve improved the error messages. Some of them don’t give enough detail. I would also create a verification script that checks all prerequisites and provides feedback about what’s missing, just like on “flutter doctor”. This would catch configuration issues before attempting a commit.”
Cross-phase	Improvement requests	Native Git integration requested	The participant wanted signing to be a core Git capability rather than a workflow assembled from separate tools.	P4, P16	2	P4 — “I would make setting up commit signing a core feature of git, rather than using a mix of tools to get it working, like during git’s initial setup.”
Cross-phase	Improvement requests	Automated or default setup requested	The participant wanted installation or signing defaults to remove manual one-time configuration.	P9, P12	2	P9 — “The setup would be improved if GPG were included with the default installation of Git. Even better, Git could be configured to sign commits by default without adding extra steps for the user.”
Sustained use	Workflow experience	Routine signing seamless or automatic	After setup, signing fit the ordinary commit workflow with little added effort.	P1, P2, P3, P4, P5, P6, P7, P8, P9, P11, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22	20	P16 — “Setting it up was way way harder. Because once it was set up, all I had to do was commit as normal, but enter a passphrase every so often. So it was super easy once it was done.”
Sustained use	Workflow experience	Setup harder than daily use	The participant explicitly contrasted difficult setup with easy ongoing signing.	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22	21	P8 — “The initial setup took around 1.5-2 hours due to troubleshooting, new concepts, and new tools. It felt a little complex for someone new to GPG and commit signing. After setting it up, signing commits was easy to do.”
Sustained use	Workflow stability	Signing process unchanged	The participant retained the same signing key, commands, or process across the semester.	P1, P2, P4, P6, P7, P8, P10, P11, P12, P13, P14, P15, P16, P17, P18, P20	16	P17 — “I did not change anything. I used the same key and process throughout the whole semester. I did make sure to verify my signatures each time though and check that even after zipping my submissions, the signature was included.”
Sustained use	Workflow adaptation	Automatic signing adopted mid-semester	The participant moved from manually supplying -S to Git configuration that signs automatically.	P9	1	P9 — “Initially, I manually added the -S flag to every commit. Then, I figured out that you can configure git to automatically do that.”
Sustained use	Key storage	Passive or default key storage	The private key remained where the tool generated or stored it without deliberate evaluation.	P1, P2, P4, P5, P8, P9, P10, P11, P14, P15, P16, P17, P18, P20	14	P18 — “For the semester, Git stored my private key for me. It was secured as much as my computer is secured, as anyone using my computer could easily access the key with Git and/or gpg commands.”
Sustained use	Key storage	Key location forgotten or unknown	The participant could not recall or determine where the private key was stored.	P1, P6, P7, P13, P18	5	P7 — “I forgot where I kept my private key. That is probably not a good thing. It was not secured with anything.”
Sustained use	Key protection	Passphrase protection	The participant reported that the private key was encrypted or gated by a passphrase/password.	P1, P3, P7, P8, P9, P10, P11, P12, P13, P15, P19	11	P8 — “The private key is encrypted by a strong passphrase, and every time I sign a commit, it asks for that passphrase. This prevents unauthorized use even if someone gains access to my computer.”
Sustained use	Key protection	Additional local security controls	The participant used restrictive permissions, encryption, or another control beyond mere default storage.	P3, P19, P20, P21, P22	5	P19 — “I stored my key pairs in the ~/.ssh directories in both my environments. They are protected by restrictive permissions. The .ssh directory as a whole has permissions of 700, meaning it’s only accessible to my user. The keys themselves are protected with permissions of 600, meaning they are not executable and only accessible by the user. The keys are also protected by a password, meaning I can’t use them without typing in my secret password.”
Sustained use	Key protection	Proactive key-management behavior	The participant intentionally managed exposure or cleanup rather than accepting defaults.	P3, P21, P22	3	P21 — “I stored my private key on my personal MacBook Pro in the GPG keyring, which is secured by my user account password. I ensured that my machine had full-disk encryption enabled through FileVault to add an extra layer of security. I did not sync this private key to cloud storage or version control; the only time I exported it was to move it to the Tesla machine for this project, and I deleted the exported files as soon as the import was complete.”
Sustained use	Future adoption	Unconditional continuation intent	The participant planned to keep signing commits without limiting use to a particular context.	P1, P2, P3, P4, P8, P9, P12, P13, P18, P19, P20, P21, P22	13	P1 — “All my git commits since I set up part 1 have been signed, so yes. Once you set it up, it’s just there. It was really easy to set up and use in everyday life, so I may as well.”
Sustained use	Future adoption	Already signing outside coursework	The participant reported already using signed commits in repositories beyond the required course assignments.	P1, P2, P3, P4, P20	5	P20 — “I can definitely see myself using signatures for my commits outside of this class, and I actually already have with my Senior Design class, due to 3 of my team members also being in this class, so we felt a need for good practice.”
Sustained use	Future adoption	Conditional continuation intent	The participant planned to sign only when the repository was sensitive, shared, professional, or otherwise high stakes.	P5, P6, P11, P14, P15, P16, P17	7	P14 — “As of now, I do not feel the need to begin signing my normal commits outside of this class. Currently, I do not create any codebases that are intended to be public, open source, or marketed as a polished product, let alone codebases that receive contributions from sourcesbesides myself; therefore, I do not feel that my repos are “high stakes” enough for signed commits to be necessary. However, I’m glad that I now have experience in creating signed commits, because I now have the option to begin making signed commits if I ever create a codebase that’s intended to be shared or published.”
Sustained use	Future adoption	No continuation due to low relevance	The participant did not plan to continue because signed commits lacked a perceived audience or need.	P10	1	P10 — “No, because nobody cares whether my commits are signed.”
Sustained use	Adoption rationale	Security or provenance rationale	Future use was justified by authenticity, integrity, provenance, or trust in authorship.	P2, P4, P8, P12, P13, P15, P19, P21, P22	9	P13 — “I plan to continue signing my commits going forward. It’s a small flex that makes one look professional on GitHub and it guarantees the provenance of my work.”
Sustained use	Adoption rationale	Professional or collaborative relevance	Future use was tied to collaboration, open-source contribution, employment, or other professional settings.	P6, P8, P12, P15, P17, P20, P21, P22	8	P12 — “Yes, I will continue to sign my commits outside of the class. I like to provide my collaborators with the confidence that I am truly the one creating commits.”
Sustained use	Adoption rationale	Personal, solo, or low-stakes repository rationale	The participant described personal, solo, private, student, or otherwise low-stakes repositories as reducing the perceived need for commit signing.	P2, P4, P10, P14, P17	5	P14 — “I do not feel that my repos are “high stakes” enough for signed commits to be necessary.”
Second device	Credential strategy	Generated a new per-device key	The participant created a new signing keypair on the second device rather than transferring the first key.	P1, P2, P6, P7, P8, P9, P11, P15, P19, P22	10	P1 — “I generated a new key pair for the new machine because I wanted practice doing this process again and I wanted there to be 2 different keys to differentiate the 2 machines.”
Second device	Credential strategy	Transferred the existing private key	The participant copied or imported the first device's private key to the second device.	P3, P4, P5, P10, P12, P13, P14, P16, P17, P18, P20, P21	12	P5 — “I decided to copy my private key on the machine because I did not want to create a new key and felt this would be simpler.”
Second device	Strategy rationale	Avoided transfer to reduce compromise risk	A new key was chosen because moving or duplicating a private key was viewed as insecure.	P6, P7, P8, P9, P19, P22	6	P8 — “I chose to generate a new key on Tesla10 rather than copying my private key from my personal computer. I chose this approach because private keys shouldn’t be transferred between machines. Each machine should have its own private key. If the private key was copied, it would’ve existed in multiple places, increasing the risk of compromise. The trade-off is that now I have two public keys to manage, but the security benefits far outweigh this small inconvenience.”

Second device	Strategy rationale	Reused key for identity continuity	The existing key was copied to preserve one signing identity or one verification relationship across devices.	P4, P18, P20, P21	4	P4 — “Furthermore, for the keys, I chose to copy them over to the Tesla machines as it was simpler to do given I had already sent in my public key for the original assignment. I could have easily used my unique ssh key on the Tesla/Hydra machines, but this would have split the keys used for the original commit and the part 2 commit. Using the same key allows me to verify all commits together, rather than worrying about two keys.”
Second device	Strategy rationale	Reused key for setup convenience	The existing key was copied to avoid creating, registering, or managing another key.	P4, P5, P10, P13, P14, P16, P17, P18, P20	9	P10 — “I copied my key pair so that I wouldn’t have to add another public key to Github”
Second device	Transfer friction	Export or import documentation gap	The participant needed additional searching because primary signing documentation omitted private-key export or import steps.	P3, P4, P10, P12, P13, P16, P17, P18, P21	9	P13 — “Once there, I was able to export the public key using instructions from GitHub’s documentation [1]. Unfortunately, I still had to export the private key which is not listed. I found what I needed with some searching and help from stackoverflow [2].”
Second device	Reconfiguration friction	Signing configuration repeated or omitted	The second device required Git/signing configuration to be recreated, and an omitted setting caused failure or rework.	P2, P5, P9, P10, P11, P20	6	P2 — “The hardest step was understanding why my allowedSignersFile was causing issues (again).”
Second device	Transfer friction	Imported key required explicit trust	An imported GPG key produced a trust warning and had to be explicitly trusted before clean verification.	P17, P21	2	P17 — “After signing, I went to verify my signature when I saw a message that read "WARNING: This key is not certified with a trusted signature!" After looking up what the issue was, I learned that I had to trust the public key. This was not needed before because the keys were generated on my computer, but imported keys must be trusted to avoid that message. Once I trusted it, I noticed there was still a message that showed it checked the trust database before signing which I thought was interesting.”
Second device	Transfer friction	Key location or shell inconsistency	The participant could not locate the key because key-listing behavior differed by shell or environment.	P13	1	P13 — “To do this, the first step I had to take was finding where GPG put the keys on my device. Running —list-secret-keys returned nothing (on PowerShell) and some time was lost poking around in the gnupg folder and testing that my commits were still in fact signing before realizing that this command would only work when run in Git Bash -- the shell I had used to set up the key to begin with. Why this a ects where the key file is stored is beyond me.”
Second device	Transfer experience	Transfer straightforward once known	After the correct method was known, moving the key files was described as easy or trivial.	P3, P4, P5, P12, P13, P14, P16, P17, P18, P20	10	P17 — “I chose to import my keys to the new machine because I thought it would be simpler. Because I have the lab machine connected to my laptop, it was a simple drag and drop to transfer the files. I do think it would have been simpler to just generate new keys, but I was curious if the signature would look different on a different machine. I also thought that knowing how to move my keys would be a useful thing to know.”
Second device	Reconfiguration friction	Full setup burden repeated	Adding a device required repeating the earlier signing setup rather than reusing a remembered configuration.	P1, P2, P3, P5, P6, P7, P8, P9, P10, P11, P13, P14, P16, P20	14	P1 — “I had to set up an SSH key for GitHub again. Then, I followed the freecodecamp.org tutorial again.”
Commit verification	Analysis strategy	CLI-only verification strategy	The participant used command-line Git tools as the sole stated strategy for commit-history analysis.	P2, P3, P5, P6, P9, P10, P12, P13, P20, P22	10	P12 — “I used the git cli.”
Commit verification	Analysis strategy	Web-only verification strategy	The participant used the GitHub web interface as the sole stated strategy for commit-history analysis.	P14, P16	2	P16 — “I just used Github's website.”
Commit verification	Analysis strategy	Hybrid CLI and web strategy	The participant used both command-line output and the GitHub web interface to assess commits.	P1, P4, P7, P8, P11, P15, P17, P19, P21	9	P19 — “For this activity, I only used the Git CLI to display the signing details for each commit and to clone the repo. I also used the GitHub web interface to verify my results by checking the verification bubbles in the commits list.”
Commit verification	Trust configuration	allowedSignersFile attempted	The participant tried to configure Git's SSH allowedSignersFile to map trusted identities to public keys.	P2, P3, P4, P5, P7, P8, P10, P11, P12, P13, P15, P19	12	P2 — “Learning from last time, I remembered I needed an allowedSignersFile in my Git configuration that contained the public key information. So, I created a file named allowed_signers (no file extension) with the same contents as id_rsa.pub and updated my Git settings.”
Commit verification	Trust configuration	Successfully configured allowedSignersFile	The participant successfully configured Git's SSH allowedSignersFile so commit signatures could be matched against trusted signer identities.	P2, P5, P7, P11, P12, P13, P15, P19	8	P2 — “Using this information, I was able to format their public keys and add it to my allowed_signers file in ~/.config/git.”
Commit verification	Trust configuration	allowedSignersFile configuration failure	The trusted-signer file could not be configured correctly or required repeated trial and error.	P3, P4, P8, P10	4	P4 — “these keys are not keys, at least in any format I know of”
Commit verification	Trust configuration	Did not successfully configure verification	The participant did not successfully configure the allowedSignersFile-based SSH signature-verification workflow, including participants who failed or did not attempt the configuration.	P1, P3, P4, P6, P8, P9, P10, P14, P16, P17, P18, P20, P21, P22	14	P4 — “They resulted in being unhelpful, as I could not get any public keys to make an allowed signers file.”
Commit verification	Trust configuration	Fingerprint confused with full public key	The participant treated supplied SHA256 fingerprints as if they were usable public-key material or did not know full keys were needed.	P3, P4, P8, P10, P13, P15, P17	7	P13 — “The main information sources I used are the blog listed as reference 5 and git’s git-log documentation [7]. The blog was useful in getting started, but the Git documentation is too laconic to be helpful. CoPilot was a mixed bag that provided suggestions for things to try out that failed, but it did identify my issue was trying to use fingerprints where full public keys were expected, which was helpful. DuckDuckGo searching was useful to find sources of information as issues came up. Unfortunately, I never found a single source that made it clear how to use and verify signatures.”
Commit verification	Mechanism confusion	GPG and SSH verification conflated	Prior GPG-signing knowledge was applied to an SSH-signed repository, producing confusing errors or an inappropriate workflow.	P8, P10, P11, P15	4	P8 — “Understanding the SSH signature verification errors. The error messages were confusing because I had only worked with GPG signatures and not SSH signatures, plus the errors didn’t really explain what was wrong or didn’t provide information on how I could fix it. It was difficult because it required me to understand multiple signature mechanisms.”
Commit verification	Abandonment	Cryptographic verification abandoned	The participant stopped trying to use keys/signatures and based the repository judgment on warning signs or other surface cues.	P4, P8, P10, P11, P15, P22	6	P4 — “I would have liked more guidance in how to verify commits using another developer's private keys, because it was difficult to figure out. Ultimately, with all the warning signs I saw, I determined the repository was not safe, but I ended up never using the keys.”
Commit verification	Alternative analysis	Manual code content inspection	The participant reviewed changed code or searched for suspicious strings in addition to or instead of verifying signatures.	P1, P6, P7, P14, P16	5	P1 — “I used git log with grep to look for API, password, key, token, eval, exec, decode, etc. phrases within the code files. I found a hit in one commit that leaked the API key and had an eval() function. I also used git log -p -S ‘’ to search for which commit and file was leaking this information.”

Commit verification	Alternative analysis	Benign content discounted signature anomaly	The participant concluded that unverified commits were not problematic because their code appeared benign.	P14	1	P14 — “At first glance, the commit history contains several suspicious commits, as nine of the 20 commits are unverified. However, I was able to manually read through all of the code in these nine commits and determine that none of them contain anything that appears to be malicious. Therefore, I do not think that any of the commits in this repository are genuinely problematic.”
Commit verification	Tool failure	Tool crash treated as repository evidence	A local Git crash or core dump was interpreted as a sign that the repository itself was corrupted or suspicious.	P22	1	P22 — “3. Git log signature inspection crashed Running: git log --show-signature --decorate --oneline resulted in: Aborted (core dumped) This is extremely unusual. It indicates possible repository corruption or malformed objects, which is itself a serious red flag.”
Commit verification	Interface preference	Web verification preferred over CLI	The participant judged GitHub's web indicators easier or more useful than command-line verification.	P1, P7, P10, P14, P16	5	P10 — “The process is fine if you just use the Github website which will show when commits are properly signed. Doing it in the CLI is a waste of time.”
Commit verification	Detected anomaly	Unknown contributor detected	A commit was flagged because its contributor was not among the authorized developers named in the task.	P3, P5, P7, P8, P12, P13, P15, P16, P17, P20, P21, P22	12	P15 — “Additionally, there was another contributor that was unmentioned in the prompt.”
Commit verification	Detected anomaly	Timestamp manipulation detected	A commit was flagged because its date was implausibly in the future or far in the past.	P1, P7, P11, P12, P13, P17, P19, P20, P21	9	P1 — “This commit is committed in 2052, it makes me think it is a modified date and not a legitimate commitment.”
Commit verification	Detected anomaly	Author and committer mismatch detected	A commit was flagged because the author and committer identities differed or the author metadata was internally inconsistent.	P1, P2, P3, P5, P11, P13, P17, P19, P20	9	P1 — “Authored by one person and committed by another.”
Commit verification	Detected anomaly	Wrong signing key detected	A commit was flagged because the signing fingerprint did not belong to the claimed committer.	P1, P2, P3, P5, P7, P11, P12, P13, P17, P19, P20	11	P19 — “There were three commits that used the wrong fingerprint for the committing user.”
Commit verification	Detected anomaly	Missing signature detected	A commit was flagged because it contained no signature or was shown as unverified due to no signature.	P2, P3, P6, P7, P9, P10, P11, P12, P13, P19, P20	11	P19 — “In total, six commits contained no signature at all, with only two of them being merge commits.”
Security reasoning	Q1: Signing properties	Authenticity or provenance benefit	Signing was understood to bind a commit to the person or key that produced it and resist impersonation.	P1, P2, P3, P4, P5, P6, P7, P8, P9, P11, P12, P13, P14, P15, P17, P18, P19, P20, P21	19	P1 — “Some benefits are authenticity and integrity. The signed commit verifies that I made the commit and not someone else impersonating me.”
Security reasoning	Q1: Signing properties	Integrity or tamper-evidence benefit	Signing was understood to reveal whether the committed content had been altered after signing.	P1, P3, P5, P8, P9, P15, P19, P20, P21	9	P1 — “It also shows whether or not a commit has been tampered with.”
Security reasoning	Q1: Signing trade-offs	Key-management or usability burden	A drawback of signing was the need to store, protect, recover, or re-establish trust in keys.	P1, P4, P5, P6, P9, P15, P17, P19, P20, P21	10	P1 — “Some drawbacks though are that users have to keep up with their keys, which can be difficult. Additionally, if someone loses their key or loses access to their key, they have to reestablish trust which can be tedious.”
Security reasoning	Q1: Signing properties	Wrong: SSH push authentication substitutes for signing	The response claimed that an SSH key used to push commits already prevents commit-author impersonation, making signing unnecessary.	P10	1	P10 — “I think it is mostly just a waste of time for most people. You already need a ssh key in most cases to push commits to Github so as long as you are the only person with the private key, no one else should be able to impersonate you on Github. Furthermore if someone has access to your Github account they could just add their own singing key in your name.”
Security reasoning	Q1: Signing properties	Misconception: poor key management negates signing	The response argued that key-management risk makes unsigned commits preferable or safer.	P7, P16	2	P16 — “I can’t think of any drawbacks? The only possible one I can think of, is if you force someone to use them, and they have to set up the keys, but aren’t very tech savvy or paying attention, they could compromise their keys? Which is more of an issue then just having unsigned commits. But this is super niche.”
Security reasoning	Q2: Multi-device keys	Synchronized key offers identity continuity	Using one key across devices was understood to preserve one identity and simplify verification or key distribution.	P1, P3, P6, P8, P12, P17, P19, P20, P21	9	P1 — “Synchronizing the key is nice because it gives you a singular identity to use.”
Security reasoning	Q2: Multi-device keys	Synchronized key increases exposure	Using one key on multiple devices was understood to increase attack surface or broaden compromise impact.	P1, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P16, P17, P18, P19, P20, P21	19	P1 — “Synchronizing the key is nice because it gives you a singular identity to use. However, it also makes you an easier target because if the key becomes compromised on one device, it’s compromised on all devices, but this also means it is easier to deactivate the key on GitHub as you are only dealing with 1 key and not multiple.”
Security reasoning	Q2: Multi-device keys	Per-device keys compartmentalize compromise	Separate keys were understood to limit a compromised key to one device or a smaller set of commits.	P1, P3, P4, P5, P6, P7, P8, P11, P12, P13, P14, P17, P18, P19, P20, P21	16	P1 — “Generating a different key is nice because only one device is compromised at a time and you can just revoke the key for the compromised device and move on. You also never have to mess around with moving your private key and accidentally leaking it or exposing it.”
Security reasoning	Q2: Multi-device keys	Per-device keys add management burden	Separate keys were understood to increase the number of credentials, public keys, or trust relationships to manage.	P1, P3, P4, P5, P6, P8, P9, P10, P11, P12, P13, P15, P16, P17, P19, P20, P21	17	P1 — “A drawback to this is that it can become a lot of keys to manage and it can be easy to lose track or lose a key. Additionally, with multiple keys you’ll have multiple verified identities on commits could get a bit complicated, especially when it comes to establishing trust.”
Security reasoning	Q3: Lost key	Replace and reconfigure after key loss	The response proposed generating a new key, registering its public key, and updating signing configuration after loss.	P1, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, P17, P18, P19, P20, P21	19	P14 — “To continue signing commits after losing a key, I would simply need to go back through the process of setting up signed commits for the first time, creating a new key and linking it back to my GitHub account, and then using that new key for commits going forward.”
Security reasoning	Q3: Lost key	Revoke or remove the lost key	The response proposed revoking, unregistering, or otherwise distrusting the lost key.	P1, P4, P6, P7, P8, P9, P11, P12, P13, P14, P16, P19, P20	13	P14 — “In this case, I should immediately revoke my lost key so that it cannot be retrieved and used for malicious commits; this should theoretically remove any security vulnerabilities that come with losing a key.”
Security reasoning	Q3: Lost key	Misconception: lost-key revocation and trust consequences unclear	The response correctly proposed generating a replacement key but expressed uncertainty about how to disable the old key and who could continue using it.	P6	1	P6 — “The old key could still be used by actors with a verified status, I suppose. I don’t know how that could be fixed—mark the key as dead somehow?”
Security reasoning	Q3: Lost key	Misconception: consequences of key loss misunderstood	The participant misunderstood an important consequence of losing a private signing key, either by claiming that prior signatures become unverifiable or by overlooking the exposure window between losing the key and revoking or distrusting it.	P3, P9, P11	3	P3 — “To continue signing commits I think you would need to create a whole new key. Because you no longer have access to your old key, your old commits may not be verifiable as your commits anymore and you cannot trust them. You also cannot go back and re-sign them with your new key. The workflow will be affected if you need those verified signatures, and you might have to go back to the commits where you knew where your private key was - losing potentially a lot of work.”
Security reasoning	Q4: Stolen key	Revoke compromised key and rotate	The response proposed key-level revocation/removal and generation/registration of a replacement key after theft.	P1, P4, P6, P7, P8, P9, P11, P19, P20, P21	10	P19 — “Similar to the previous question, I would unregister my public key with GitHub. Then, I would generate and register a new key pair. Finally, I would examine recent commits on my repos to see if malicious code was added using my old fingerprint.”

Security reasoning	Q4: Stolen key	Audit or notify after key theft	The response proposed reviewing recent commits or alerting collaborators/users about the compromised key.	P4, P6, P7, P8, P9, P11, P13, P14, P17, P19, P20, P21	12	P14 — “If I were to have my key stolen, I should immediately revoke the key so that it no longer has the ability to sign verified commits, and then I should manually go through my account’s commit logs to make sure that no commits were made using the key in the time since it was stolen. This still leaves room open for human error (I could miss a malicious commit that had been well-hidden), but this should theoretically solve any security issues from having a key stolen.”
Security reasoning	Q4: Stolen key	Wrong: account action substitutes for revocation	The response treated creating a new account or repository as the remedy without addressing the compromised key itself.	P15, P16	2	P16 — “I think the biggest thing would be either like, disabling them in GitHubs key settings? Or maybe make a new account if one has been compromised.”
Security reasoning	Q4: Stolen key	Misconception: secure storage substitutes for revocation	The response correctly recognized the impersonation risk from a stolen key but proposed improved key storage without explicitly revoking or distrusting the compromised key.	P5	1	P5 — “To mitigate this, use tools that help store keys.”
Security reasoning	Q4: Stolen key	Misconception: revocation and manual audit fully resolve key theft	The response correctly recommended revocation and auditing but overstated these measures as resolving all security consequences of the theft, despite possible missed malicious commits and incomplete propagation of revocation.	P14	1	P14 — “This should theoretically solve any security issues from having a key stolen.”
Security reasoning	Q2: Multi-device keys	Misconception: signing-key reuse is inherently bad	The response transferred a general rule against private-key reuse to the repeated use of a long-term signing key.	P15	1	P15 — “For synchronizing an existing key, I think it could be bad to reuse keys. We have been told repeatedly that it is bad to reuse private keys, so this might apply to signing keys as well.”
Security reasoning	Q5: Attack vectors	Credential theft or social engineering	Attackers were expected to phish, steal credentials/keys, or take over a maintainer account.	P1, P3, P4, P5, P7, P8, P10, P11, P14, P15, P16, P19, P21	13	P5 — “An attacker could convince a developer to add a key to their repo under the identity of someone else which would allow them to push malicious code. They could also try and steal a developers account and take over their commits.”
Security reasoning	Q5: Attack vectors	Malicious contribution or trusted-insider path	Attackers were expected to submit a malicious pull request/change or first gain trust as a contributor before inserting harmful code.	P1, P3, P6, P7, P8, P9, P11, P12, P13, P17, P21	11	P21 — “An attacker might try to compromise open source project repositories by attempting to gain access to the maintainers’ signing keys or by submitting malicious pull requests.”
Security reasoning	Q5: Defenses	Code review or multiple approvals	The response recommended human review, multiple reviewers, or approval before merge.	P1, P3, P5, P6, P7, P8, P9, P13, P15, P17, P18, P19, P20, P21	14	P15 — “To defend against these attacks, the developers should make Git commit signing mandatory, frequently monitor the repository, and require multiple reviewers for merges.”
Security reasoning	Q5: Defenses	Repository or account access controls	The response recommended controls such as MFA, least privilege, limited merge access, or defense in depth.	P4, P7, P8, P10, P11, P13, P14, P16, P17, P18, P19, P20	12	P8 — “A potential attack could be an attacker might try to steal developer credentials through phishing, malware, or credential stuffing. An attacker could inject malicious code into a project’s dependencies, or an attacker could impersonate a trusted developer. My recommendation for security would be to require multiple reviews and to never merge with a single approval. Another recommendation that I would provide would be to have mandatory commit signing. Adding access control, which means limiting the number of people with merge access. However, no single practice is sufficient, and defense in depth is essential.”
Security reasoning	Q5: Defenses	Require signing and verify signatures	The response recommended signed commits and/or checking signatures against authorized identities.	P1, P3, P5, P6, P8, P9, P11, P15, P18, P19, P20, P21	12	P21 — “An attacker might try to compromise open source project repositories by attempting to gain access to the maintainers’ signing keys or by submitting malicious pull requests. To reduce this risk, I would recommend requiring signed commits from maintainers, using code review processes, and the normal security practices used in open source projects.”